

ReactJS

Understanding Props and PropTypes in React:

Let's first understand the difference between export default vs named export in React.

What are exports?

In React, we use exports to share code from one file to another. There are two main types:

1. export default
2. export (named export)

What is 'export default'?

What it means: A file can have only ONE default export

It is used when:

- You want to export main thing from a file

Example: a basic example.

Create a new file named, Module1.mjs:

```
src > JS Module1.mjs > [🔗] default
1  const a = "Aditya";
2  const b = "Utsav";
3  const c = "Pihu";
4
5  export default a;
```

And one another file named, Modul2.mjs:

```
src > JS Module2.mjs
1  import any_name from './Module1.mjs'
2  console.log(any_name);
```

Now, in terminal try to run the Module2.mjs using the node:

```
PS D:\ReactJS-revisited\my-app\src> node Module2.mjs
Aditya
PS D:\ReactJS-revisited\my-app\src> █
```

What this output indicates about the 'export default'?

It exports one main value from a file. Even though you had 3 variables (a, b, c), only this one was exported: *export default a;* (look at line 5 of Module1.mjs). Also, we can name the import anything you want, here we wrote 'import any_name' but still got the 'Aditya'. So this shows: default export is NOT bound to a name.

Only ONE default export is allowed per file, if one write like this in Module1.mjs:

```
src > JS Module1.mjs > [🔍] default
1   const a = "Aditya";
2   const b = "Utsav";
3   const c = "Pihu";
4
5   export default a;
6   export default b;
```

And then try to run the node: it will give the error.

```
PS D:\ReactJS-revisited\my-app\src> node Module2.mjs
file:///D:/ReactJS-revisited/my-app/src/Module1.mjs:6
export default b;
      ^^^^^^^^^

SyntaxError: Identifier 'default' has already been declared
    at compileSourceTextModule (node:internal/modules/esm/utils:344:16)
    at ModuleLoader.moduleStrategy (node:internal/modules/esm/translators:105:18)
    at #translate (node:internal/modules/esm/loader:534:12)
    at ModuleLoader.loadAndTranslate (node:internal/modules/esm/loader:581:27)

Node.js v22.16.0
PS D:\ReactJS-revisited\my-app\src> |
```

Now, let's talk about named exports.

What are 'named export'?

What it means: You can export multiple things from a file.

Example: Module1.mjs:

```
src > JS Module1.mjs > ...
1   export const a = "Aditya";
2   export const b = "Utsav";
3   export const c = "Pihu";
```

See use of the
'export'
keyword.

While Module2.mjs:

```
src > JS Module2.mjs
1   import { a, b, c } from './Module1.mjs';
2
3   console.log(a);
4   console.log(b);
5   console.log(c);
```

Now, in the terminal:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS D:\ReactJS-revisited\my-app\src> node .\Module2.mjs
Aditya
Utsav
Pihu
PS D:\ReactJS-revisited\my-app\src> |
```

What you learn from this output about 'named export'?

- Names are FIXED in named exports, we must use the exact same names.
- You can import only what you need
- Multiple exports are allowed (unlike default)
- You can rename while importing (using 'as')

```
src > JS Module2.mjs
1  import { a as name1, b as name2 } from './Module1.mjs';
2
3  console.log(name1);
4  console.log(name2);|
```

How does they differ from each other?

Feature	Named Export	Default Export
How many per file	Many	Only 1
Import syntax	{ a }	anyName
Name flexibility	Fixed	Flexible
Common use	utilities, multiple values	main component/value

Can we use both together?

Yes, we can use both 'export default' and 'named export' together.

Example:

Module1.mjs:

```
src > JS Module1.mjs > ...
1  const a = "Aditya";
2  const b = "Utsav";
3  const c = "Pihu";
4
5  export default a;
6  export {b};
```

Module2.mjs:

```
src > JS Module2.mjs
1  import xyz, {b} from './Module1.mjs';
2
3  console.log(xyz);
4  console.log(b);
```

Terminal:

```
PS D:\ReactJS-revisited\my-app\src> node .\Module2.mjs
Aditya
Utsav
PS D:\ReactJS-revisited\my-app\src> 
```

Now, let's talk about the "Props"!

What are Props?

Props = Properties. In simple words: Props are used to pass data from one component to another in React.

Think like:

- Function arguments
- Input to a component

Why Props are needed?

Without props:

- Every component shows same data
- No flexibility

With props:

- We can send different data to same component

Example: a basic example without props (using no extra file)

App.js:

```
src > JS App.js > App
1  function Greeting(){
2      return(
3          <>
4          <h1>Hello</h1>
5          </>
6      )
7  }
8
9  function App() {
10     return (
11         <>
12         <Greeting/>
13         <Greeting/>
14         </>
15     );
16 }
17
18 export default App;
```

Terminal:

```
PS D:\ReactJS-revisited\my-app> npm start

You can now view my-app in the browser.

  Local:            http://localhost:3001
  On Your Network:  http://192.168.1.7:3001

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

Output:

A screenshot of a web browser's address bar. It features navigation icons (back, forward, refresh) on the left, an information icon in the center, and the text 'localhost:3001' on the right.

Hello

Hello

Now, the same example using the props:

App.js:

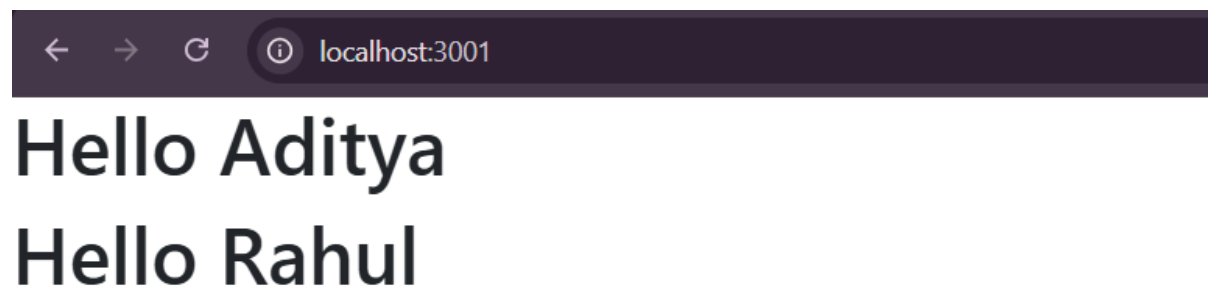
```
src > JS App.js > [🔍] default
1  function Greeting(props) {
2    return <h1>Hello {props.name}</h1>;
3  }
4
5  function App() {
6    return (
7      <div>
8        <Greeting name="Aditya" />
9        <Greeting name="Rahul" />
10     </div>
11   );
12 }
13
14 export default App;
```

Receiving Data

name = "Aditya" is a prop

Using data

Output:



Can we use props to pass other datatypes except string?

Yes, we can do so.

- String → "text"
- Number → {20}

Example:

App.js:

```
src > JS App.js > [🔍] App
1  function Greeting(props) {
2    return <h1>Hello {props.name} Age: {props.age}</h1>;
3  }
4
5  function App() {
6    return (
7      <div>
8        <Greeting name="Aditya" age={20}/>
9        <Greeting name="Rahul" age={16}/>
10     </div>
11   );
12 }
13
14 export default App;
```

Output:

Hello Aditya Age: 20

Hello Rahul Age: 16

What if the default behaviour of props?

App.js: (here we just emptied the passed value, it is not default behaviour example)

```
src > JS App.js > default
1  function Greeting(props) {
2    return <h1>Hello {props.name} Age: {props.age}</h1>;
3  }
4
5  function App() {
6    return (
7      <div>
8        <Greeting name="Aditya" age={20}/>
9        <Greeting name="Rahul" age={} />
10     </div>
11   );
12 }
13
14 export default App;
```

Output:

Compiled with problems:

ERROR in ./src/App.js

Module build failed (from ./node_modules/babel-loader/lib/index.js):
SyntaxError: D:\ReactJS-revisited\my-app\src\App.js: JSX attributes must only be assigned a non-empty expression. (9:33)

```
7 |   <div>
8 |     <Greeting name="Aditya" age={20}/>
> 9 |     <Greeting name="Rahul" age={} />
    |                                     ^
10 |   </div>
11 | );
12 | }
    at constructor (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:365:19)
    at FlowParserMixin.raise (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:6616:19)
    at FlowParserMixin.jsxParseAttributeValue (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:4618:16)
    at FlowParserMixin.jsxParseAttribute (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:4665:38)
    at FlowParserMixin.jsxParseOpeningElementAfterName (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:4679:28)
    at FlowParserMixin.jsxParseOpeningElementAfterName (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:4880:18)
    at FlowParserMixin.jsxParseOpeningElementAt (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:4674:17)
    at FlowParserMixin.jsxParseElementAt (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:4698:33)
    at FlowParserMixin.jsxParseElementAt (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:4710:32)
    at FlowParserMixin.jsxParseElement (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:4761:17)
    at FlowParserMixin.parseExprAtom (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:4771:19)
    at FlowParserMixin.parseExprSubscripts (D:\ReactJS-revisited\my-app\node_modules\@babel\parser\lib\index.js:11098:23)
```

Example: default behaviour. Props have no built-in default values. If missing → undefined.

App.js:

```
src > JS App.js > App
1  function Greeting(props) {
2    |   return <h1>Hello {props.name} Age: {props.age}</h1>;
3    | }
4
5  function App() {
6    |   return (
7    |     <div>
8    |       <Greeting/>
9    |       <Greeting/>
10   |     </div>
11   |   );
12   | }
13
14   export default App;
```

Output:

```
← → ↻ ⓘ localhost:3001
```

Hello Age:

Hello Age:

How to handle the missing prop of above example?

We can use default props to do so.

Example:

App.js: (modern way)

```
src > JS App.js > default
1  function Greeting({name="Guest"}) {
2    |   return <h1>Hello {name}</h1>;
3    | }
4
5  function App() {
6    |   return (
7    |     <div>
8    |       <Greeting/>
9    |       <Greeting/>
10   |     </div>
11   |   );
12   | }
13   export default App;
```


Output:

Hello Guest

Example: Default inside function body

App.js:

```
src > JS App.js > [🔍] default
1  function Greeting(props) {
2    |   const name = props.name || "Guest";
3    |   return <h1>Hello {name}</h1>;
4    | }
5
6  function App() {
7    |   return (
8    |     <div>
9    |       | <Greeting/>
10   |     </div>
11   |   );
12   | }
13
14  export default App;
```

Output:

Hello Guest

Example: defaultProps (older approach)

App.js:

```
src > JS App.js > ...
1  function Greeting(props) {
2    |   return <h1>Hello {props.name}</h1>;
3    | }
4
5  Greeting.defaultProps = {
6    |   name: "Guest"
7    | };
8
9  function App() {
10   |   return (
11   |     <div>
12   |       | <Greeting/>
13   |     </div>
14   |   );
15   | }
16
17  export default App;
```

Output:

Discouraged

Now, let's understand about passing JSX with props:

How to pass HTML code via JSX?

children is the recommended React pattern for passing markup from a parent component to a child component.

Example:

Sample.jsx:

```
src > myComponent > Sample.jsx > default
1  import React from 'react'
2
3  function Sample({children}) {
4    return (
5      <div style={{color:"green", border:"5px solid green", width:"300px"}}>
6        {children}
7      </div>
8    )
9  }
10
11 export default Sample
```

App.jsx:

```
src > App.jsx > App
1  import React from 'react'
2  import Sample from './myComponent/Sample'
3  function App() {
4    return (
5      <div>
6        <Sample>
7          <h1> Hello, Aditya</h1>
8        </Sample>
9      </div>
10    )
11  }
12
13 export default App
```

Output:

Hello, Aditya

Now, suppose we want to make the colour of the text dynamic as well:

App.jsx:

```
src > App.jsx > ...
1  import React from 'react'
2  import Sample from './myComponent/Sample'
3  function App() {
4    return (
5      <div>
6        <Sample color="Pink">
7          <h1> Hello, Aditya</h1>
8        </Sample>
9        <Sample>
10         <h1> Hello, Utsav</h1>
11       </Sample>
12     </div>
13   )
14 }
15
16 export default App
```

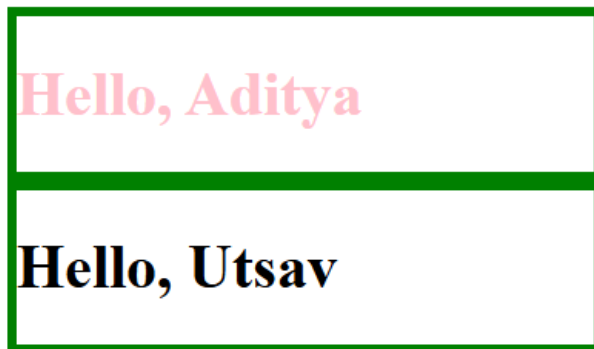
In this code, the App component renders the Sample component twice. The first Sample receives a color prop with the value "Pink" and displays the heading **"Hello, Aditya"** as its children, so the text appears in pink. The second Sample is rendered without a color prop and displays **"Hello, Utsav"** as its children. Since both headings are passed between the opening and closing <Sample> tags, they are received through the children prop and rendered inside the <div> defined in the Sample component. This demonstrates how React components can be reused with different content and props to produce different outputs.

Sample.jsx:

```
src > myComponent > Sample.jsx > Sample
1  import React from 'react'
2
3  function Sample({children, color}) {
4    return (
5      <div style={{color:color, border:"5px solid green", width:"300px"}}>
6        {children}
7      </div>
8    )
9  }
10
11 export default Sample
```

Sample is a reusable React functional component that accepts two props: children and color. The children prop represents any JSX content placed between the <Sample> opening and closing tags when the component is used, and it is rendered inside the <div>. The color prop is used to dynamically set the text color through inline styling. The component also applies a fixed green border and a width of 300px, making it a simple wrapper that can display different content and text colors based on the props passed to it.

Output:



Now, suppose we want to make **Get Input Field Value**, I meant suppose we want to see the text user entered dynamically below the text box.

App.jsx:

```
src > App.jsx > App
1  import React, { useState } from 'react'
2
3  function App() {
4    const [val, setVal] = useState("Aditya");
5    return (
6      <div>
7        <h1>Get input field value:</h1>
8        <input type='text' onChange={(event) => setVal(event.target.value)} placeholder='Type here' />
9        <h1>{val}</h1>
10      </div>
11    )
12  }
13
14  export default App
```

Output:

Get input field value:

Aditya

Output: when we begin typing in the box

Get input field value:

yes I am

This code demonstrates how to use React's `useState` hook to capture and display the value entered into an input field. The state variable `val` is initialized with "Aditya". Whenever the user types in the input box, the `onChange` event is triggered, and `setVal(event.target.value)` updates the state with the current input value. Since React re-renders the component whenever state changes, the `<h1>{val}</h1>` element immediately displays the latest text entered by the user. This creates a real-time connection between the input field and the displayed value, a pattern known as a **controlled component** in React.

Now, suppose we want that the value in both the input field and the text displayed below it get deleted when we click on "clear" button, then we can use this:

App.jsx:

```
src > App.jsx > App
1  import React, { useState } from 'react'
2
3  function App() {
4    const [val, setVal] = useState("Aditya");
5    return (
6      <div>
7        <h1>Get input field value:</h1>
8        <input type='text' value={val} onChange={(event)=>setVal(event.target.value)} placeholder='Type here' />
9        <h1>{val}</h1>
10       <button onClick={()=>setVal("")}> Clear </button>
11     </div>
12   )
13 }
14
15 export default App
```

Output:

Get input field value:

Utsav

Output: when clicked on the button

Get input field value:

Understanding Controlled Componenta:

What are Controlled Components?

A controlled component in React is a form element (such as an `<input>`, `<textarea>`, or `<select>`) whose value is controlled by React state rather than the DOM itself.

Example:

App.jsx:

```
src > App.jsx > ...
1  import React, { useState } from 'react';
2
3  function App() {
4    const [val, setVal] = useState('');
5    return (
6      <div>
7        <h1>Aditya</h1>
8        <input
9          type="text"
10         value={val}
11         onChange={(e) => setVal(e.target.value)}
12       />
13        <p>{val}</p>
14      </div>
15    );
16  }
17
18  export default App;
```

Output:

Aditya

Output: when started typing something

Aditya

Utsav

How it works

1. The input's value is stored in React state (val).
2. The value attribute is linked to that state.
3. When the user types, onChange updates the state.
4. React re-renders the input with the updated value.

This means React is the single source of truth for the input value.

What are Uncontrolled Components?

An uncontrolled component is a form element whose value is managed by the DOM itself instead of React state. React does not track the input value on every keystroke. Instead, you access the value only when needed, typically using a ref.

In other words:

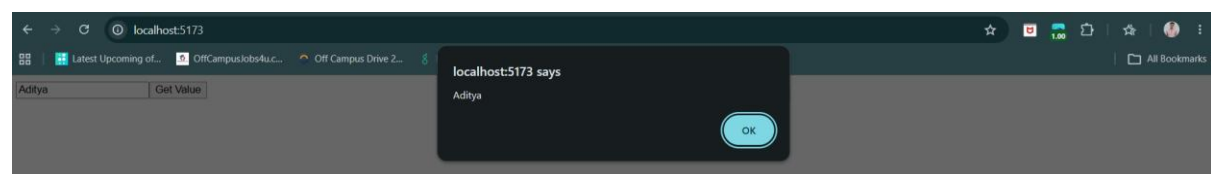
- Controlled Component → React state controls the input value.
- Uncontrolled Component → The DOM controls the input value.

App.jsx:

```
src > App.jsx > App
1  import React, { useRef } from 'react';
2
3  function App() {
4    const inputRef = useRef();
5    const handleClick = () => {
6      alert(inputRef.current.value);
7    };
8    return (
9      <div>
10       <input
11         type="text"
12         ref={inputRef}
13         placeholder="Enter your name"
14       />
15       <button onClick={handleClick}>
16         Get Value
17       </button>
18     </div>
19   );
20
21   export default App;
```

Output:

Output: when typed something and clicked on the “Get Value” button.



How It Works

1. `useRef()` creates a reference to the input element.
2. The input stores its own value in the DOM.
3. React does not update state when the user types.
4. When the button is clicked, `inputRef.current.value` retrieves the current value from the DOM.

When to Use Uncontrolled Components

- Simple forms with minimal validation.
- Integrating with non-React libraries.
- File inputs (`<input type="file">`), which are commonly handled as uncontrolled components.

Feature	Controlled Component	Uncontrolled Component
Source of Truth	React state	DOM itself
Data Handling	Form data is managed by React using <code>useState</code>	Form data is managed by the browser/DOM
Value Access	Through state variables	Through refs (<code>useRef</code>) or DOM queries
Input Value	Set using the <code>value</code> prop	Set using <code>defaultValue</code> or left unmanaged
Change Handling	Uses <code>onChange</code> to update state	No need to update state on every change
Validation	Easy to validate in real time	Usually validated when submitting
Predictability	More predictable and easier to debug	Less predictable because React doesn't track changes
Performance	Slightly more re-renders (state updates on every change)	Fewer re-renders
Code Complexity	More code	Less code
Recommended By React	Generally preferred for most forms	Useful for simple forms or integrating non-React code

Understanding State & Handling Events in React :

First understand about the Hooks.

What are Hooks?

In React, Hooks are special functions that let you use state and other React features in functional components (without writing class components).

In brief:

- **`useState`** → lets you add and manage state
- **`useEffect`** → handles side effects (like API calls, timers)
- **`useContext`** → shares data across components easily

What is State in React?

State is data inside a component that can change over time.

Why do we need State?

Without state:

- UI is static
- No updates happen

With state:

- UI updates automatically when data changes

Example: a basic example without the useState.

App.js:

```
src > JS App.js > [🔗] default
1  function App() {
2    let fruit = "Apple";
3    return (
4      <>
5        <h1>State in React JS</h1>
6        <h1>{fruit}</h1>
7      </>
8    );
9  }
10
11 export default App;
```

Output:

State in React JS

Apple

Now, suppose I want that there be a button and when someone clicks on that button the name of the fruit will get changed. For this, we will definitely need a function, and the button.

App.js:

```

src > JS App.js > [🔗] default
1  function App() {
2      let fruit = "Apple";
3      const handleFruit=()=>{
4          fruit = "Banana";
5      }
6      return (
7          <>
8          <h1>State in React JS</h1>
9          <h1>{fruit}</h1>
10         <button onClick={handleFruit}>Click to change the fruit name</button>
11         </>
12     );
13 }
14
15 export default App;

```

Output:

State in React JS

Apple

Click to change the fruit name

Output: when button was clicked.

State in React JS

Apple

Click to change the fruit name

But, why this doesn't worked? Like, we have given the function, said that when this function is used, the value of fruit to be changed to the banana, but still Apple is reflected. Why?

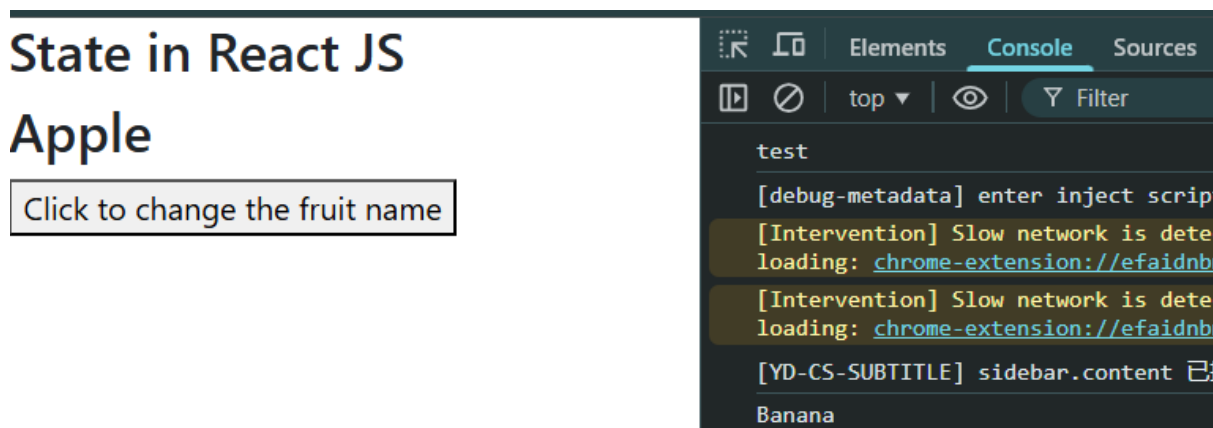
ReactJS shows the update on the UI only when its component get re-rendered.

Now, To prove that value is updated but not reflected in the UI we will console the updated value.

App.js:

```
src > JS App.js > App > handleFruit
1 function App() {
2   let fruit = "Apple";
3   const handleFruit={()=>{
4     fruit = "Banana";
5     console.log(fruit)
6   }}
7   return (
8     <>
9     <h1>State in React JS</h1>
10    <h1>{fruit}</h1>
11    <button onClick={handleFruit}>Click to change the fruit name</button>
12    </>
13  );
14 }
15
16 export default App;
```

Output:



Clearly, in console it says that function was called, and value of fruit is updated from Apple to Banana and reflected in console, but not in the UI.

So, how to overcome this? We will use State.

Let's talk about 'state' again.

What is State?

State in React:

- State is a built-in feature used to store and manage data inside a component.
- It is mutable, meaning it can change over time.
- When state changes, React automatically re-renders the component to update the UI.
- In functional components, state is managed using the useState hook.

Example:

```
const [count, setCount] = useState(0);
```

Here:

- count → current state value
- setCount → function used to update state

- `useState(0)` → initial value is 0

How does state differ from variable?

Feature	Variable	State (useState)
Part of	JavaScript	React
UI update	No	Yes (auto re-render)
Mutability	Yes	Yes (via setter function)
Used in UI	Not reliable	Yes (recommended)

How to import the state?

In React, we don't import "state" directly. Instead, we import the Hook called `useState` from React.
Correct way:

`import { useState } from "react";`

Example: using `useState` for that example of updating the fruit name. We are still not updating.

App.js:

```

src > JS App.js > App
1  import { useState } from "react";
2
3  function App() {
4    const [fruit, setFruit] = useState("Apple");
5    return (
6      <>
7      <h1>State in React JS</h1>
8      <h1>{fruit}</h1>
9      </>
10   );
11 }
12
13 export default App;

```

importing

Current state value

This is a function used to update the state

useState is React hook, while "Apple" is the initial state value

Output:

State in React JS

Apple

Does useState always returns an array?

Yes, useState always returns an array in React. What it returns:

- Current state value
- Function to update that state

Now, we will implement the part of button.

App.js:

```
src > JS App.js > [🔗] default
1  import { useState } from "react";
2
3  function App() {
4    const [fruit, setFruit] = useState("Apple");
5    const handleFruit = () =>{
6      setFruit("Banana");
7    }
8    return (
9      <>
10     <h1>State in React JS</h1>
11     <h1>{fruit}</h1>
12     <button onClick={handleFruit}>Click</button>
13   </>
14 );
15 }
16
17 export default App;
```

Output:

State in React JS

Apple

Click

Output: when clicked.

State in React JS

Banana

Click

In this React code, `useState("Apple")` creates a state variable `fruit` with the initial value "Apple" and a function `setFruit` to update it. When the button is clicked, the `handleFruit` function runs and calls `setFruit("Banana")`, which changes the state from "Apple" to "Banana". As soon as the state updates, React automatically re-renders the component, so the updated value is shown on the screen.

Now, we will see the example of counter.

Counter.js: (no update logic)

```
src > JS Counter.js > [?] default
1  import { useState } from "react";
2
3  function Counter(){
4    //Taking a state
5    const [count, setCount] = useState(0);
6
7
8    return(
9      <>
10     <h1> Counter: {count} </h1>
11     <button>Update Counter</button>
12     </>
13   )
14 }
15
16 export default Counter;
```

Output:

Counter: 0

Update Counter

Now, writing the update logic.

Counter.js:

```
src > JS Counter.js > Counter
1  import { useState } from "react";
2
3  function Counter() {
4    //Taking a state
5    const [count, setCount] = useState(0);
6    return (
7      <>
8        <h1> Counter: {count} </h1>
9        <button onClick={() => setCount(count + 1)} >Update Counter</button>
10      </>
11    )
12  }
13
14  export default Counter;
```

Output: initially

Counter: 0

Update Counter

Output: when clicked

Counter: 1

Update Counter

Now, we are writing the same update the counter logic, but this time as a separate function.

Counter.js:

```
src > JS Counter.js > Counter
1 import { useState } from "react";
2
3 function Counter() {
4   //Taking a state
5   const [count, setCount] = useState(0);
6   const updateCounter = () =>{
7     setCount(count+1);
8   }
9   return (
10    <>
11      <h1> Counter: {count} </h1>
12      <button onClick={updateCounter}>Click</button>
13    </>
14  )
15 }
16
17 export default Counter;
```

Output: at start

Counter: 0

Click

Output: when button was clicked.

Counter: 1

Click

Now, we can even use two states in the same code. We will see that in the counter example as well, where the other one be to count in reverse.

App.js:

```
src > JS Counter.js > Counter > rCounter
1  import { useState } from "react";
2
3  function Counter() {
4    //Taking a state
5    const [count, setCount] = useState(0);
6    const [rcount, setRevCount] = useState(10);
7    const updateCounter = () =>{
8      setCount(count+1);
9    }
10   const rCounter = () =>{
11     setRevCount(rcount-1);
12   }
13   return (
14     <>
15       <h1> Counter: {count} </h1>
16       <button onClick={updateCounter}>Click</button>
17       <br></br>
18       <h1> RevCounter: {rcount} </h1>
19       <button onClick={rCounter}>Click to Reverse count</button>
20     </>
21   )
22 }
23
24 export default Counter;
```

Output: at start

Counter: 0

Click

RevCounter: 10

Click to Reverse count

Output: when clicked.

Counter: 4

Click

RevCounter: 5

Click to Reverse count

This React component uses two state variables: count initialized to 0 and rcount initialized to 10. Clicking the first button increases count by 1, while clicking the second button decreases rcount by 1 using their respective state update functions. Whenever state changes, React automatically re-renders the component and updates the UI with the new values.

Example: using useState to change the status of the text from “On” to “off”.

LightSwitch.js:

```
src > JS LightSwitch.js > default
1  import { useState } from "react";
2
3  function LightSwitch(){
4
5      const [isOn, setIsOn] = useState(false);
6      const toggleLight={()=>{
7          setIsOn(!isOn);
8      }}
9      return(
10         <>
11         <h1>The light is {isOn?"On":"off"}</h1>
12         <button onClick={toggleLight}>Click to change the status</button>
13         </>
14     )
15 }
16
17 export default LightSwitch;
```

Output: at start

The light is off

Click to change the status

Output: when button was clicked.

The light is On

Click to change the status

This React component creates a simple light switch using the useState Hook. The variable isOn stores whether the light is currently ON (true) or OFF (false), and it starts as false, meaning the light is initially OFF. When the button is clicked, the toggleLight function runs and changes the state to the opposite value using !isOn (true becomes false, and false becomes true). The <h1> element displays "On" when isOn is true and "off" when it is false. React automatically re-renders the component whenever the state changes, so the displayed status updates instantly when the button is clicked.

Example: using useState to Show/Hide Text Component.

ShowHideText.js:

```
src > JS ShowHideText.js > [?] default
1  import { useState } from "react";
2  function ShowHideText(){
3      // useState
4      const[show, setShow] = useState(true);
5      // function to toggle
6      const toggleText= () => {
7          setShow(!show);
8      }
9      return(
10         <>
11         <button onClick={toggleText}>{show?"Hide":"Show"}</button>
12         {show && <p>Hello! I am visible now</p>}
13         </>
14     )
15 }
16 export default ShowHideText;
```

Output:

Show

Output: when clicked

Hide

Hello! I am visible now

It uses useState to store a value called show, which starts as true, meaning the text is visible at the beginning. When the button is clicked, the toggleText function runs and changes show to the opposite value using !show (so true becomes false and false becomes true). The button text also changes dynamically—showing “Hide” when the text is visible and “Show” when it is hidden. Finally, the paragraph is displayed only when show is true using show && <p>...</p>, so clicking the button hides and shows the message instantly without reloading the page.

Example: to capitalise the text entered by the user.

CapitalizeText.js:

```
src > JS CapitalizeText.js > [🔍] default
1  import { useState } from "react"
2  function CapitalizeText() {
3      const [text, captext] = useState("Enter your text");
4      const cap = () => {
5          captext(text.toUpperCase());
6      }
7      return (
8          <>
9              <input
10                 value={text}
11                 onChange={e => captext(e.target.value)}
12                 placeholder="Enter your text"
13             />
14             <button onClick={cap}>Click to capitalize</button>
15         </>
16     )
17 }
18 export default CapitalizeText;
```

Output:

Enter your text	Click to capitalize
-----------------	---------------------

Output: when something was typed and button was clicked.

MY NAME IS ADITYA	Click to capitalize
-------------------	---------------------

This code defines a controlled component in React that manages user input and performs a string transformation. By invoking the `useState` hook, it initializes a state variable (`text`) and its corresponding setter function (`captext`). The `<input>` element is bound to this state via its `value` attribute, creating a one-way data binding from state to view. To achieve two-way data binding, an `onChange` event handler captures user input events, extracting the real-time value via `e.target.value` and updating the state, which triggers a component re-render. When the button is clicked, it executes the `cap` event handler, which utilizes the JavaScript `toUpperCase()` prototype method to transform the current state string and commits the mutated, capitalized string back to the state, causing the DOM to update with the final capitalized text.

Example: using useState to change the theme from light to dark and viceversa.

ToggleMode.js:

```
src > JS ToggleMode.js > ToggleMode
1  import { useState } from "react";
2  function ToggleMode() {
3      const [mode, setMode] = useState(false);
4      const changeMode = () => {
5          setMode(!mode);
6      };
7      const pageStyle = {
8          backgroundColor: mode ? "black" : "white",
9          color: mode ? "white" : "black",
10         height: "100vh",
11         padding: "20px"
12     };
13     return (
14         <>
15         <div style={pageStyle}>
16             <h2>Current Mode: {mode ? "Dark" : "Light"}</h2>
17             <button onClick={changeMode}>Click to change</button>
18         </div>
19         </>
20     );
21 }
22
23 export default ToggleMode;
```

Output: by default

Current Mode: Light

Click to change

Output: when button was clicked.

Current Mode: Dark

Click to change

This code builds a webpage that flips between light and dark themes when you click a button. Inside the component, a boolean (true/false) memory switch called mode is created using useState(false). When the button is clicked, it runs the changeMode function, which simply flips that switch to its opposite value. Because the styles of the page and the text on the screen are tied directly to this switch using conditional shortcuts, the entire background color and text instantly swap from light to dark (or dark to light) every time the button is pressed.

Now, keep in mind these things about the component:

App.js: (We can have more than one component in a file, and we can use one component inside another by calling it. The component we export (App) becomes the main component available outside the file.)

```
src > JS App.js > [🔍] default
1  function App(){
2      return(
3          <>
4          <h1>App component</h1>
5          <Greeting/>
6          </>
7      )
8  }
9  function Greeting(){
10     return(
11         <>
12         <h2>Greeting component</h2>
13         </>
14     )
15 }
16
17 export default App;
```

Output:

App component

Greeting component

In this code, App is the main component that is exported and rendered. Inside App, an `<h1>` heading and the Greeting component are displayed. The Greeting component is another component in the same file that returns an `<h2>` heading. When App renders, it also renders Greeting, so both headings appear on the screen. The empty tags (`<>` `</>`) are React Fragments, which let us group multiple elements without adding an extra HTML element.

Also, keep in mind that we can do like this in JSX:

```
src > JS App.js > App
1 function App(){
2   let name = "Aditya";
3   let x = 10;
4   let y = 20;
5   return(
6     <>
7     <h1>My name is {name}</h1>
8     <h1>x+y = {x+y}</h1>
9     <h1>20+40 = {20+40}</h1>
10    </>
11  )
12 }
13 export default App;
```

Output:

My name is Aditya

x+y = 30

20+40 = 60

Also, keep in mind that we can use the functions inside the component.

App.js:

```
src > JS App.js > App
1 function App(){
2   function fruit(){
3     return "Apple";
4   }
5   return(
6     <>
7     <h1>{fruit()}</h1>
8     </>
9   )
10 }
11 export default App;
```

Output:

Apple

Now, let's understand how to handle the checkboxes in react:

App.jsx:

```
src > App.jsx > [default]
1  import { useState } from "react";
2  function Skills() {
3    const [skills, setSkills] = useState([]);
4    const handleSkills = (event) => {
5      const { value, checked } = event.target;
6      console.log(value, checked);
7      if (checked) {
8        setSkills([...skills, value]);
9      } else {
10       setSkills(skills.filter((skill) => skill !== value));
11     }
12   };
13   return (
14     <div>
15       <h3>Select Your Skills</h3>
16       <input onChange={handleSkills} type="checkbox" id="php" value="PHP" /> <label htmlFor="php">PHP</label>
17       <input onChange={handleSkills} type="checkbox" id="java" value="Java" /> <label htmlFor="java">Java</label>
18       <input onChange={handleSkills} type="checkbox" id="react" value="React" /> <label htmlFor="react">React</label>
19       <h4>Selected Skills:</h4>
20       <p>{skills.join(", ")}</p>
21     </div>
22   );
23 }
24 export default Skills;
```

Output:

Select Your Skills

☐ PHP ☐ Java ☐ React

Selected Skills:

Output: when clicked on the check boxes

Select Your Skills

☒ PHP ☐ Java ☒ React

Selected Skills:

PHP, React

This React component lets users select multiple skills using checkboxes. The skills state stores all selected skills in an array. When a checkbox is checked or unchecked, the handleSkills function runs and gets the checkbox's value (e.g., "PHP") and checked status (true or false). If checked, the skill is added to the skills array using the spread operator (...skills); if unchecked, it is removed using filter(). Finally, skills.join(", ") displays all selected skills as a comma-separated list on the screen.

Now, let's understand how to handle the radio buttons in react:

App.jsx:

```
src > App.jsx > ...
1  import { useState } from "react";
2
3  function App() {
4    const [gender, setGender] = useState("male");
5    return (
6      <div>
7        <h1>Handle Radio and Dropdown</h1>
8        <h4>Select Gender</h4>
9        <input type="radio" name="gender" value="male" checked={gender === "male"} onChange={(event) => setGender(event.target.value)}
10         id="male"/>
11        <label htmlFor="male">Male</label>
12
13        <input type="radio" name="gender" value="female" checked={gender === "female"} onChange={(event) => setGender(event.target.value)}
14         id="female"/>
15        <label htmlFor="female">Female</label>
16        <h2>Selected Gender: {gender}</h2>
17      </div>
18    );
19  }
20  export default App;
```

Output:

Handle Radio and Dropdown

Select Gender

☒ Male ☐ Female

Selected Gender: male

Output: when clicked on the other radio button.

Handle Radio and Dropdown

Select Gender

☐ Male ☒ Female

Selected Gender: female

This React code creates a simple gender selection form using radio buttons. It uses the `useState` hook to store the selected gender, with "male" set as the default value. When the user clicks either the Male or Female radio button, the `onChange` event updates the gender state with the selected value. The `checked` property ensures that the correct radio button remains selected based on the current state. Finally, the selected gender is displayed on the screen in real time below the radio buttons.

Loop in JSX with Map function:

App.jsx:

```
src > App.jsx > App
1  function App() {
2    const userData = [
3      { id: 1, name: "Rahul", email: "rahul@gmail.com", age: 22 },
4      { id: 2, name: "Priya", email: "priya@gmail.com", age: 21 },
5      { id: 3, name: "Amit", email: "amit@gmail.com", age: 23 }
6    ];
7    return (
8      <div>
9        <h1>Loop in JSX with Map Function</h1>
10       <table border="1">
11         <thead>
12           <tr>
13             <td>Id</td><td>Name</td><td>Email</td><td>Age</td>
14           </tr>
15         </thead>
16         <tbody>
17           {userData.map((user) => (
18             <tr key={user.id}>
19               <td>{user.id}</td><td>{user.name}</td><td>{user.email}</td><td>{user.age}</td>
20             </tr>
21           ))}
22         </tbody>
23       </table>
24     </div>
25   );
26 }
27 export default App;
```

Output:

Loop in JSX with Map Function

Id	Name	Email	Age
1	Rahul	rahul@gmail.com	22
2	Priya	priya@gmail.com	21
3	Amit	amit@gmail.com	23